

بسمه تعالی

آزمایش ششم آزمایشگاه معماری کامپیوتر

ALU

روزبه معینی 401106544

احسان محترم 401106458

دانشگاه صنعتی شریف

بهار 1405

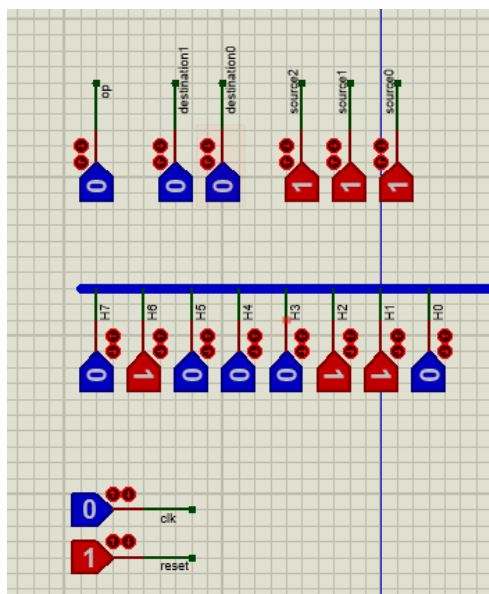
فهرست

۱. مقدمه	3
۲. طراحی داخل proteus	4
ساختار کلی مدار	4
زیربخش‌ها	4
بخش REGITSERFILE	5
بخش MUX	6
بخش Adder	7
۳. بررسی و تست در شبیه‌سازی	8
سری اول: لود چند مقدار داخل سه رجیستر با استفاده از لود موازی خط 7	8
سری دوم: بررسی چند عملیات سری	9

۱. مقدمه

هدف از این گزارش، مستندسازی مراحل انجام شده در آزمایش شماره 6 آزمایشگاه معماری کامپیوتر است. در این تمرین، به پیاده‌سازی یک جمع‌کننده/تفریق‌کننده که به یک رجیستر فایل متصل است می‌پردازیم. این مدار، بر اساس کامندهای 6 بیتی ورودی، عملیات را روی عملوند A و عملوند دوم؛ که با سه بیت مبدا مشخص می‌شود؛ انجام می‌دهد و در رجیستر مشخص شده در 2 بیت مقصد در کامند می‌نویسد.

نحوه دادن ورودی در تصویر 1 مشخص شده است. بیت اول از راست مشخص‌کننده نوع عملیات جمع یا تفریق است. دو بیت بعدی شماره رجیستر نهایی که جواب در آن سیو می‌شود را نشان می‌دهد. سه بیت اول سورس عملوند دوم عملیات را مشخص می‌کند. 000 تا 011 برای 4 رجیستر رجیسترفایل، 100 101 و 110 به ترتیب مقادیر ثابت 0 1 و -1 هستند. برای ورودی 111 نیز یک خط لود موازی در نظر گرفتیم تا بتوان مقدار اولیه در رجیسترها به کمک آن ثبت کرد. 8 بیتی که در ادامه آمده به همین ورودی متصل است و در نهایت سیگنال‌های کلاک و ریست را داریم. ریست active-low است و با یک کلاک زدن یک مرتبه پاسخ عملیات در رجیستر مقصد سیو می‌شود.



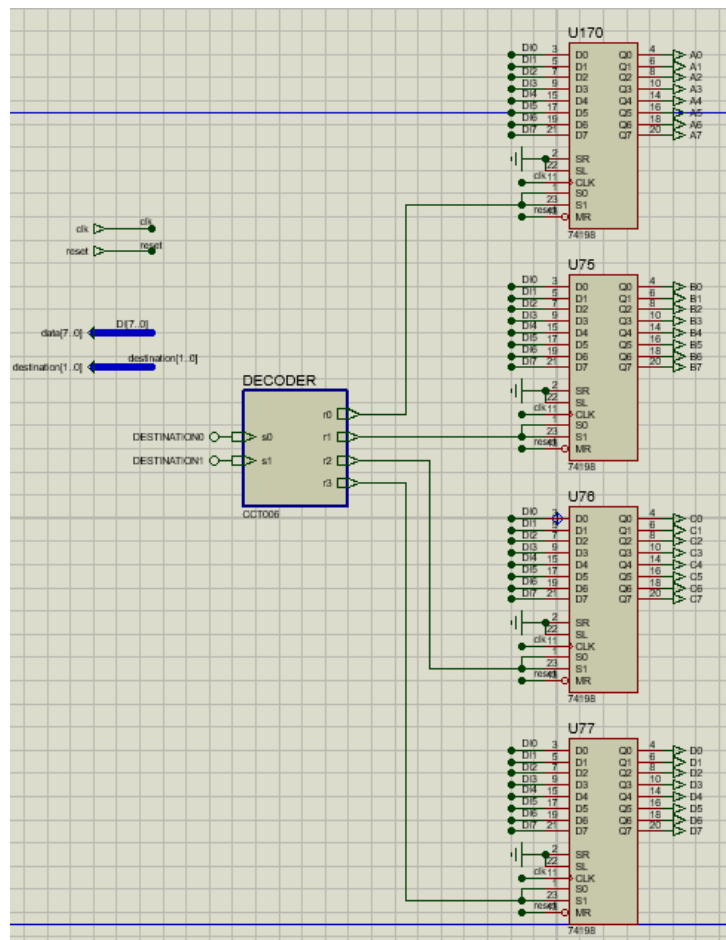
شکل 1. ساختار ورودی

- ماژول REGITSERFILE: شامل 4 رجیستر 8 بیتی با قابلیت لود از سیگنال ورودی data به رجیستر مشخص شده در سیگنال destination

- ماژول MUX: واحد انتخاب عملوند دوم بر اساس 3 بیت ورودی source. مقادیر 0، 1 و 1- برای اندیس‌های 4، 5 و 6 هاردکد شده‌اند و ورودی موازی 7 برای مقدار دهی اولیه در نظر گرفته شده‌است.
- ماژول ADDER: عملیات انتخابی جمع یا تفریق را روی عملوند اول A و عملوند دوم خروجی مالتی‌پلکسر انجام می‌دهد و نتیجه را خروجی می‌دهد.

بخش REGITSERFILE

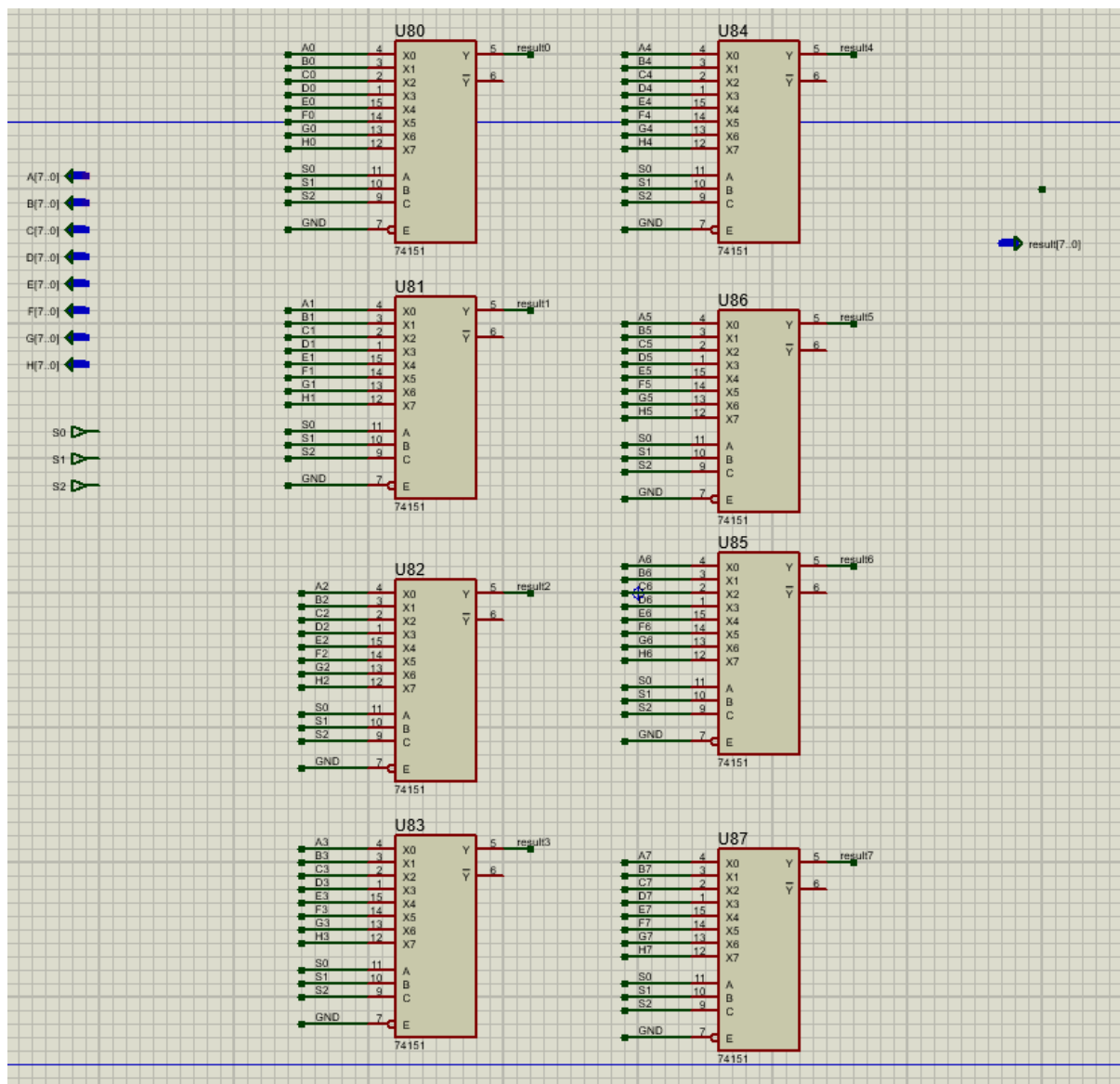
این ماژول وظیفه نگهداری مقادیر 4 رجیستر A تا D و write back پاسخ عملیات بر اساس دوبیت آدرس مقصد را برعهده دارد. ساختار مدار شامل یک دیکودر و 4 رجیستر از نوع 74198 است. وقتی دوبیت s0 و s1 در ورودی این رجیستر 1 شوند، مقادیر ورودی لود می‌شوند. پس باس دیتا را به ورودی وصل کردیم و دو بیت آدرس مقصد را به دیکودر و 4 خروجی دیکودر را به آی سی 74198 متناظر هر رجیستر. با آمدن کلاک، مقدار نوشته شده روی باس دیتا، روی رجیستر مشخص شده از خروجی دیکودر نوشته می‌شود. (شکل 3)



شکل 3. بخش نگهداری نما

بخش MUX

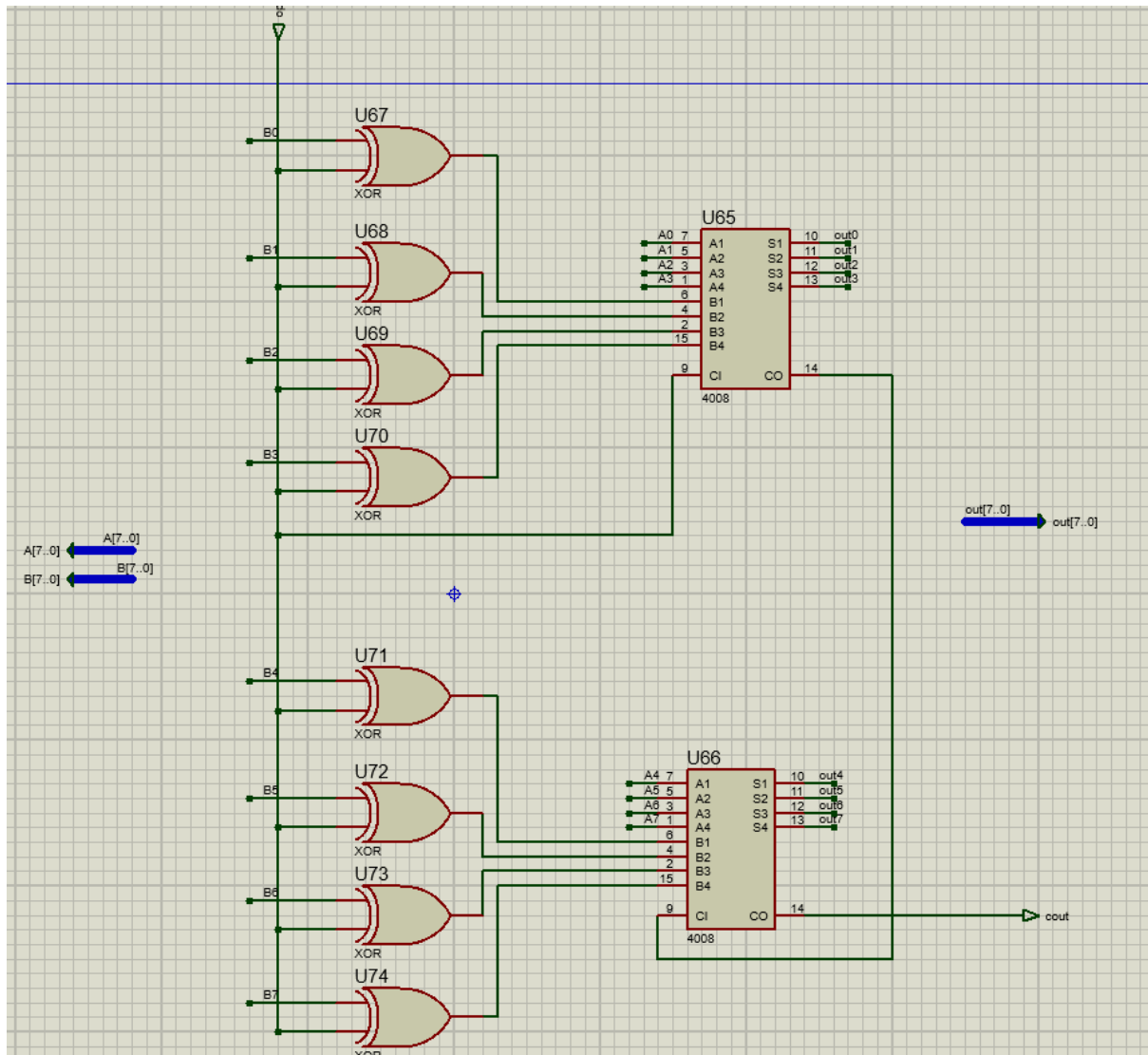
این بخش تعیین میکند عملوند دوم از بین 4 رجیستر و مقادیر ثابت کدام باشد. ساختار بسیار ساده است و برای هر بیت، بین 8 ورودی با یک آی سی 74151 که یک مالتی پلکسر 8 به یک است، خروجی متناظر آدرس را انتخاب کرده ایم. پس مجموعاً 8 آی سی 74151 عملوند را از بین 8 انتخاب برمیدارد و خروجی می دهد. (شکل 4)



شکل 4. 8 تا 74151 برای انتخاب هر بیت از عملوند دوم

بخش Adder

با استفاده از دو جمع‌کننده 4 بیتی از نوع 4008، دو عملوند ورودی را جمع کردیم. صرفاً اگر op تفریق باشد، به علت XOR شدن با تک تک مقادیر عملوند دوم و اضافه شدن به عنوان بیت نقلی ورودی، فرآیند محاسبه مکمل دوم عملوند دوم صورت می‌پذیرد. (شکل 5)



شکل 5-1. بخش شیف و نگهدارنده مانتیس

بدین ترتیب کل ساختار مدار آماده استفاده می‌شود.

۳. بررسی و تست در شبیه‌سازی

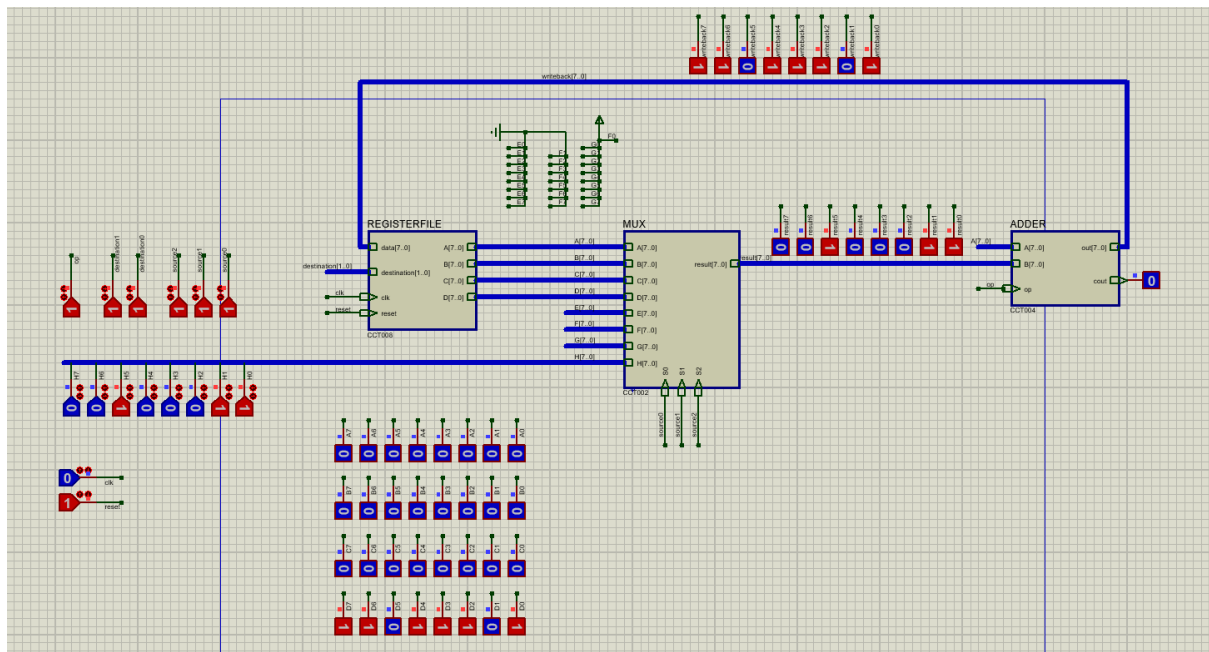
نکته مهم: نمایش مقادیر میانی باس‌ها صرفاً جهت دیباگ اضافه شده‌بودند و میتوان آن‌ها را اصلاً حذف کرد. باید توجه داشت چون MUX و ADDER مدار ترکیبی هستند، پاسخشان صرفاً بر اساس مقادیر کنونی رجیسترها و دستور تنظیم می‌شود. اما مقدار پاسخ نهایی بعد از زدن کلاک داخل رجیسترفایل نوشته می‌شود. پس با زدن کلاک اگر مقدار رجیستری عوض شود و به آن رجیستر در MUX رفرنس داشته باشیم، مقدار باس‌های میانی بلافاصله به مقادیر جدید آپدیت می‌شود.

برای تست یک مجموعه دستور را به صورت پشت هم اجرا کرده‌ایم.

سری اول: لود چند مقدار داخل سه رجیستر با استفاده از لود موازی خط 7

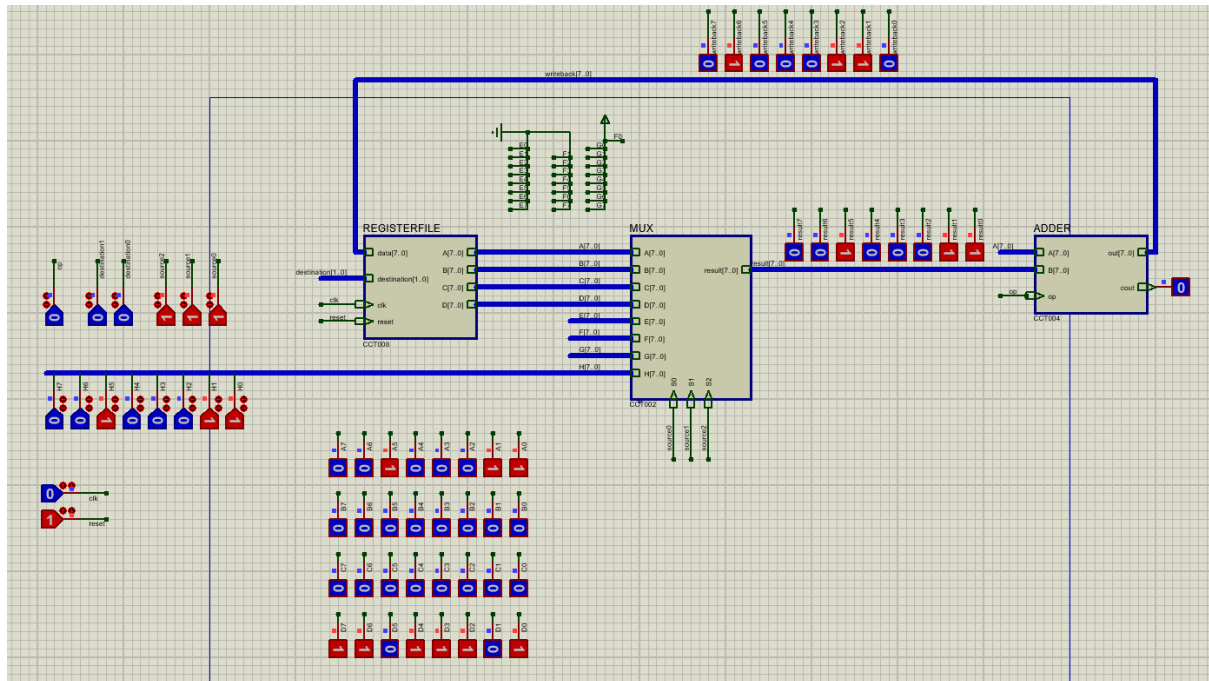
در تست اول سراغ مقدار 35 را روی خط 7 نوشتیم و یکبار با اپراند 0 و یک، مقدار 35 و -35 را روی رجیسترهای A, D لود کردیم.

- 111111 / ورودی 35 = منفی 35 یعنی مقدار 11011101 - 0 داخل رجیستر 11 یعنی D با موفقیت لود می‌شود. (شکل 6-1)



شکل 6-1. تست تفريق با صفر (لود منفی)

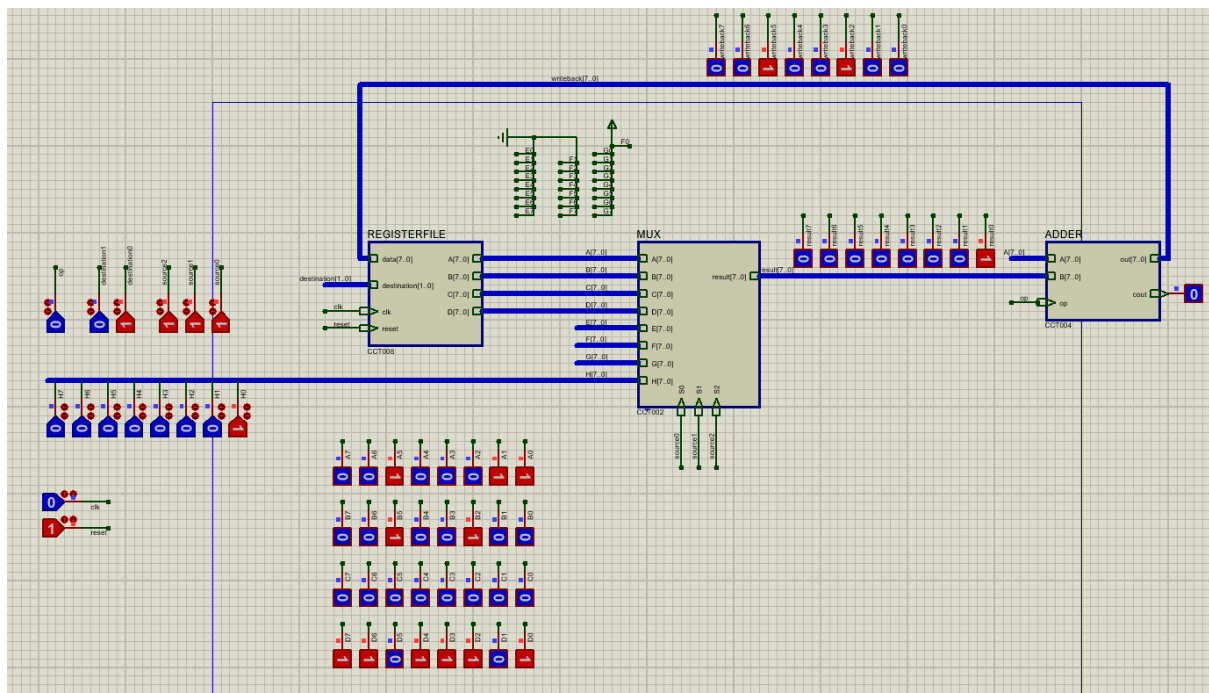
- 000111 / ورودی 35 = رجیستر A با مقدار مثبت 35 یعنی 0010011 + 0 لود شد. (2-6)



شکل 2-6. تست جمع با صفر (لود مثبت)

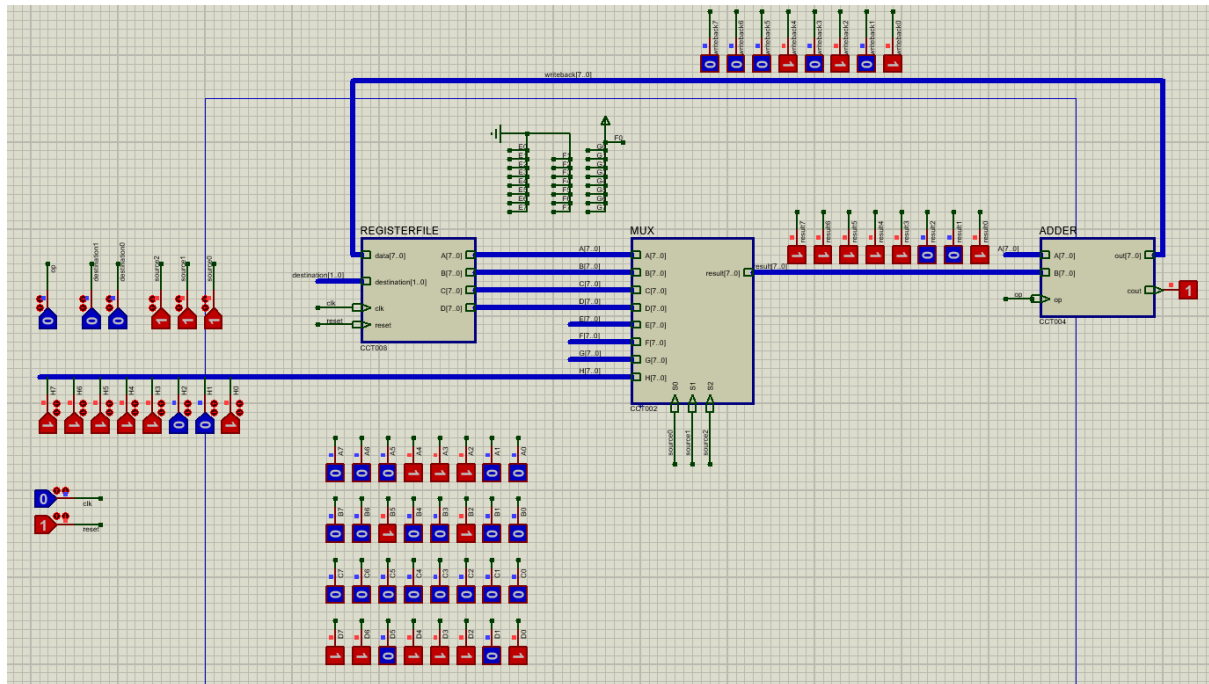
سری دوم: بررسی چند عملیات سری

- op: 001111 => B = A + Load(00000001) = 35 + 1 = 36 = 00100100 (شکل 3-6)



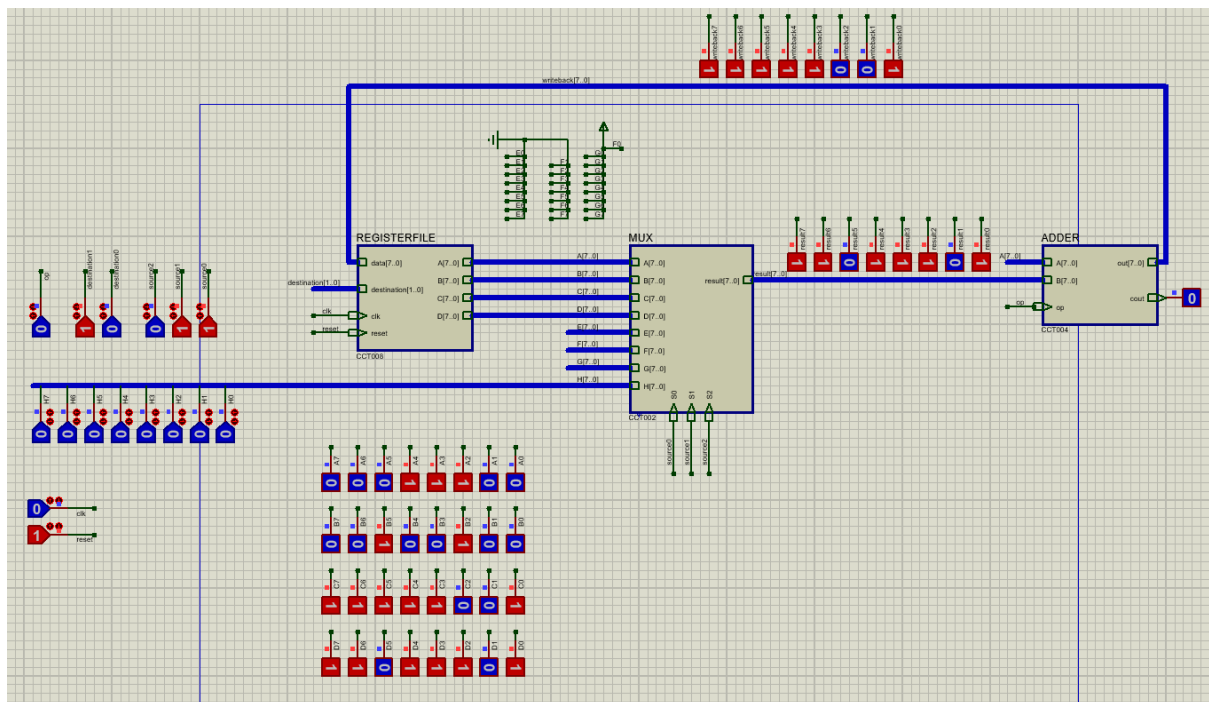
شکل 3-6. تست جمع اول

- op: 000111 => A = A + Load(11111001) => A = 35 + (-7) = 28 = (0001110)
(شكل 4-6)



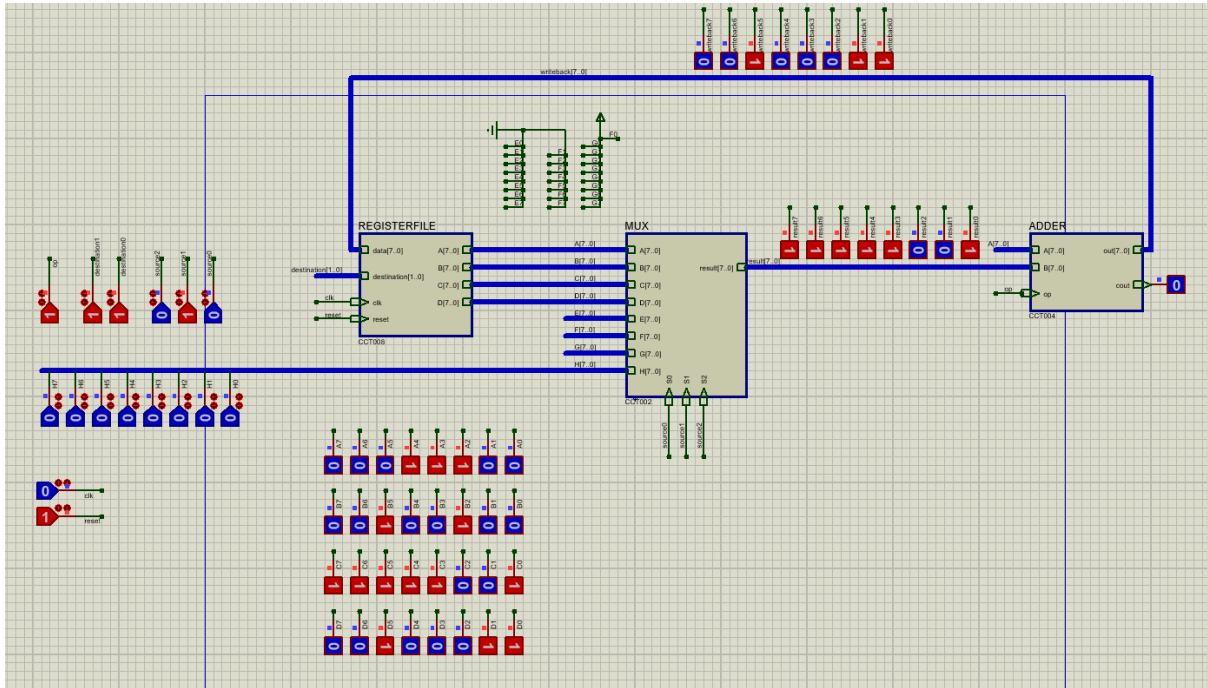
شكل 4-6. تست جمع دوم

- op: 01001 => C = A + D => C = 28 + (-35) = -7 = (11111001) (شكل 5-6)



شكل 5-6. تست جمع سوم

- op: 111010 => D = A - C => D = 28 - (-7) = 35 = (00100011) (شكل 6-6)



شكل 6-6. تست تفريق